DAY 1

下午

数据 · 可视化 · 获取

下午 · 洗干净，看明白，取得回来

Day 1 下午 · 180 min · 先清洗，再可视化，最后溯源取数

连享会《AI 数据分析实战专题》

主讲：杨楷

深圳大学 · 经济学院

00

开场

下午的三件事

下午要做的三件事

- **接着上午：上午看懂了数据为什么会脏，下午先把它洗干净**
- 然后画图：一张图顶一千行数字，学会用图发现模型看不见的问题
- 最后溯源：搞清楚这些数据是怎么从网上取回来的
- 顺序是刻意的——先看懂，再去取

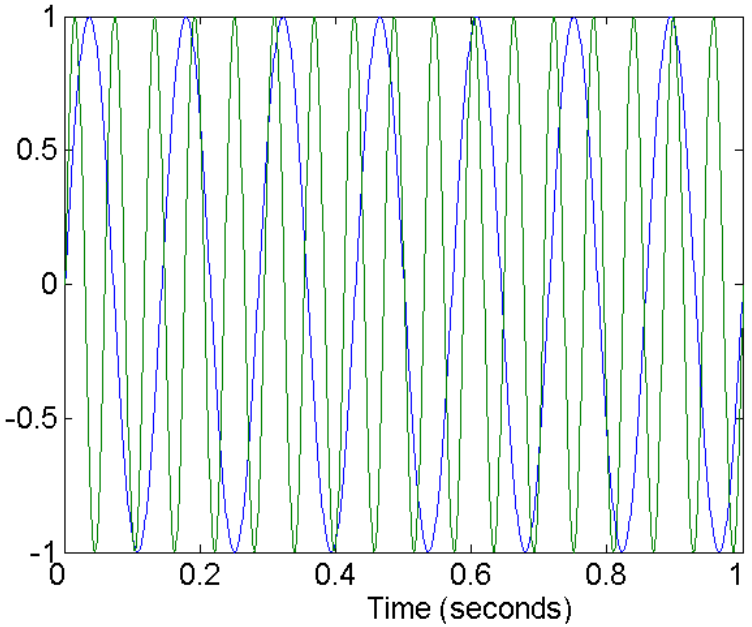
01

数据清洗

接上午：把脏数据洗干净

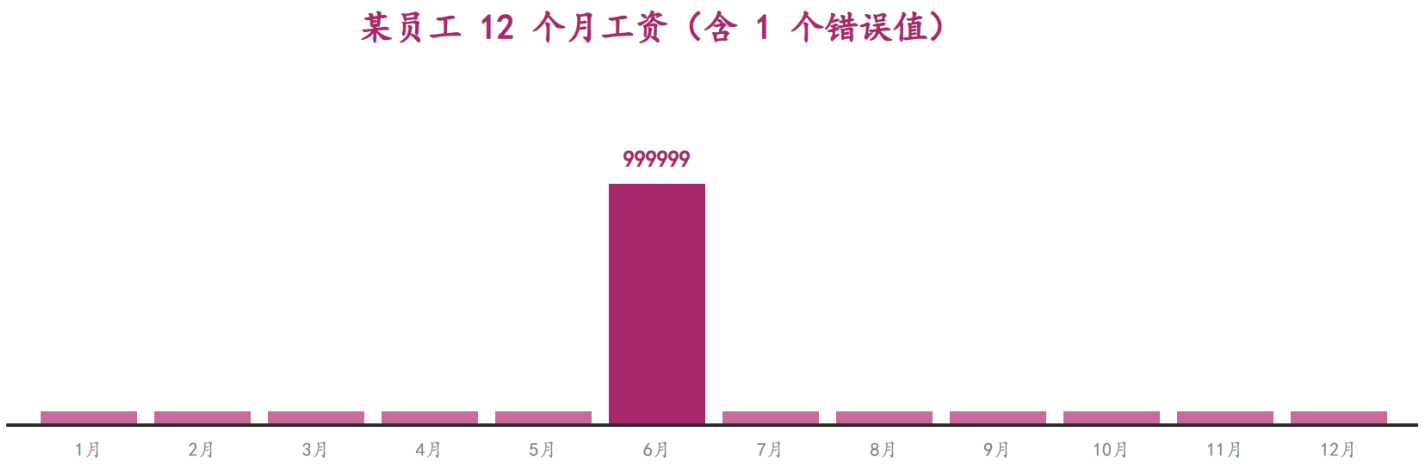
六类脏数据，上午见过

- 缺失 / 异常 / 重复 / 不一致 / 噪声 / 偏差
- 上午看懂数据为什么会脏
- 下午逐个动手解决 —— 30 秒回顾，不重讲



真实数据长什么样：一份员工表

- **找找看：空工资、负年龄、重复项、性别三种写法**
- 真实数据一上手就是脏的
- 不洗就用 = 直接用垃圾建模



均值被拉到 ≈ 90k，中位数仍是 8k —— 一个极端值毁掉均值

检测：3σ / 箱线图 → 处置：修正为 8k（或删除）→ 均值恢复正常

缺失值：空着不管还是填

- **三策略：忽略元组 / 手动填写 / 自动填充**
- 本项目：缺失不许静默删，先标注再处理
- 财务里「缺失」本身常常就是信号

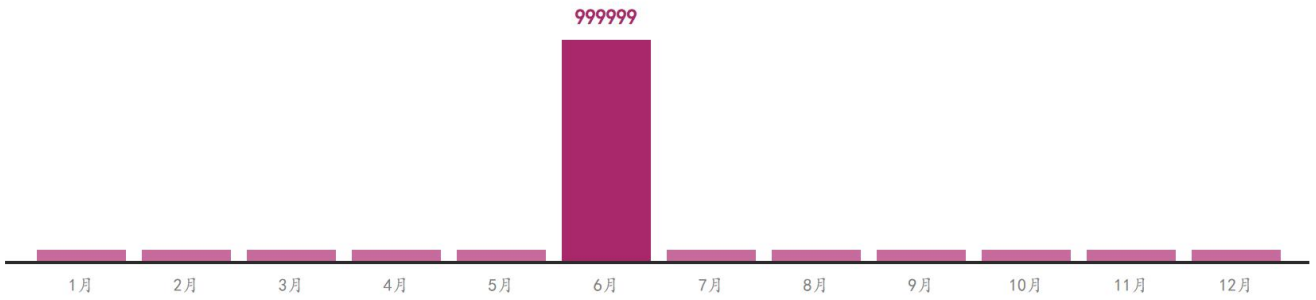


噪声：一个 999999 如何毁掉均值

案例连接：舞弊数据集的利润表里就有这种异常值 —— 先判断是错误还是舞弊线索

- **三步：检测 → 判断 → 处置**
- 大额异常先别删，它可能是舞弊线索
- 先判断是错误还是真相，再决定删不删

某员工 12 个月工资（含 1 个错误值）



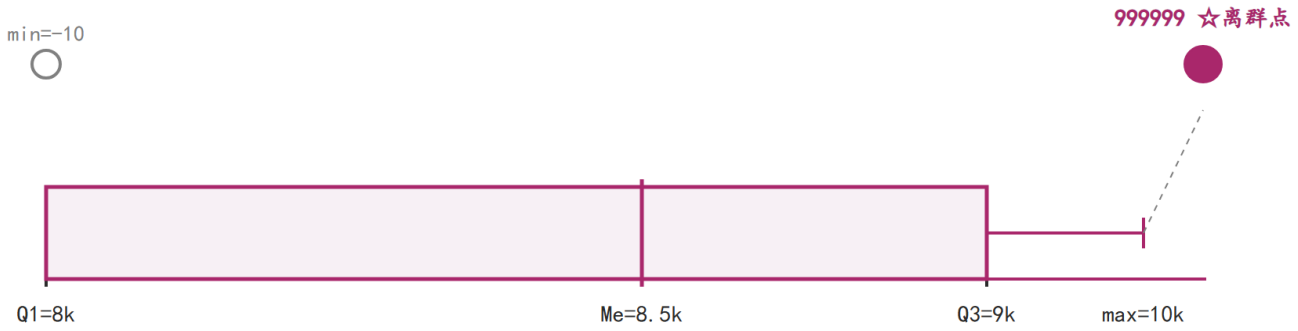
均值被拉到 ≈ 90k，中位数仍是 8k — 一个极端值毁掉均值

检测：3σ / 箱线图 → 处置：修正为 8k（或删除）→ 均值恢复正常

离群点：3 倍标准差法则

- μ ±1σ 约 68%，μ ±2σ 约 95%，μ ±3σ 约 99%
- 3σ 之外才算离群
- 箱线图是另一个工具，画图模块会回来

箱线图：5 数概括一眼看出离群点



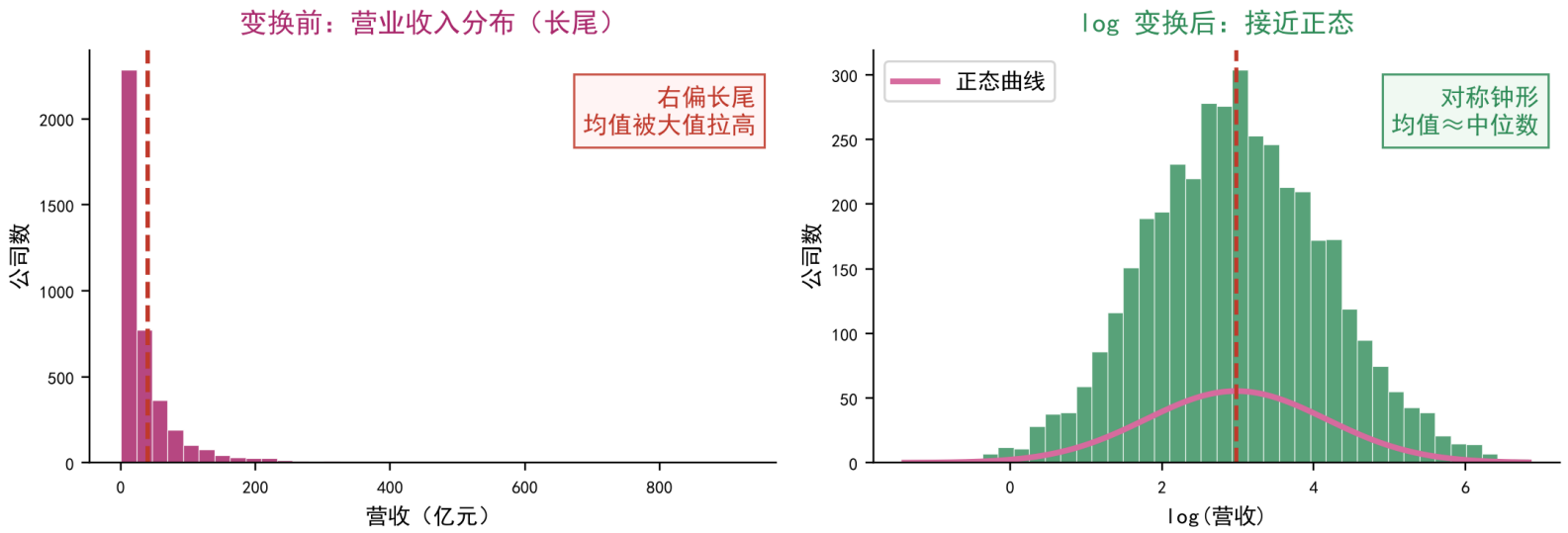
经验法则：约 99% 的数据落在 μ ±3σ 内，之外的视为离群点

审计视角：异常交易往往是离群点 → 不是删，而是"值得查的线索"

数据变换：把量纲拉到可比

- **营业收入、资产规模都是典型长尾**
- log 变换把长尾拉成接近正态
- z-score 让不同量纲的指标可以放一起看

量纲拉到可比：一次 log 变换，长尾变钟形



分类变量不能直接当数字：One-hot

- 深圳=0 广州=1 上海=2 → 模型以为上海 > 广州
- One-hot：每个取值单独一列 0/1
- 兑现上午讲的：标称变量必须这样处理

One-hot：分类列 → 多个 0/1 列

原始表

客户	城市
甲	深圳
乙	广州
丙	上海

→ One-hot

变换后

客户	深圳	广州	上海
甲	1	0	0
乙	0	1	0
丙	0	0	1

为什么不能直接编 0/1/2？2>1 会暗示"上海>广州"

One-hot 不引入大小关系 —— 代价：3 个城市 → 3 列（维度翻倍 → 需要归约）

多源合并：模式、实体与冲突

- **三源合一：财务数据 + 基本信息 + 年报文本**
- 模式集成 / 实体识别 / 冲突解决
- 同名不同表以哪张为准，事先定死写进规则

数据集成：两张表怎么合成一张

客户表

cust-id	姓名	城市
1001	张三	深圳
1002	zs	深圳

订单表

cust-#	订单号	金额
1001	ORD001	500
1001	ORD002	300

① 模式集成：cust-id = cust-#（字段名不同、含义相同）

② 实体识别：张三 = zs（同一人两种写法）

③ 冲突检测：同一客户两个城市的记录怎么处理

合并结果：客户 5 列 × 订单 3 列 → 按主键拼成一张完整宽表

案例连接：上午的舞弊数据集，利润表+资产负债表+违规表三源合一就是这个流程

为什么 80% 的时间在洗数据

- **数据科学 = 80% 清洗 + 20% 建模**
- Agent 时代大量清洗可交给 AI 代劳
- 但规则得你写 —— 写清楚，AI 才不会替你做决定



02

数据可视化

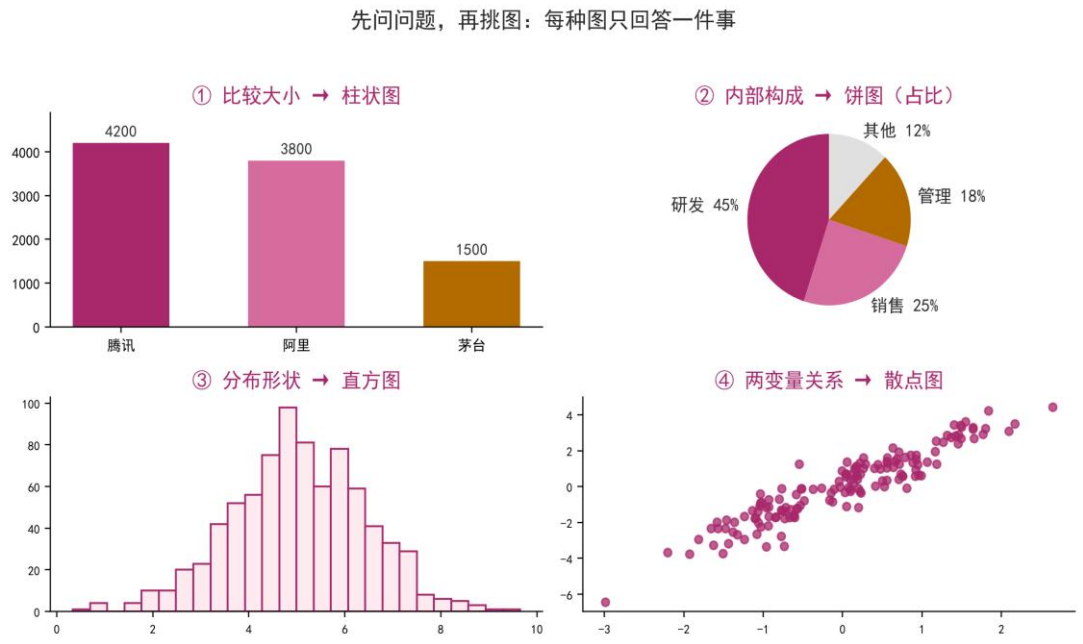
看图选型 • 读懂分布 • 发现异常

为什么要先画图再建模

- **建模之前先看数据长什么样，能省掉后面无数返工**
- 分布偏不偏、缺得多不多、有没有离谱的值
- 探索阶段的图是给自己看问题的
- 装饰报告是后面的事，现在先别管好看

图选错了，等于什么都没说

- **比较 → 柱状图；构成 → 堆叠图**
- 分布 → 直方图；关系 → 散点图
- 先问问题，再挑图 —— 这张速查表拍照保存



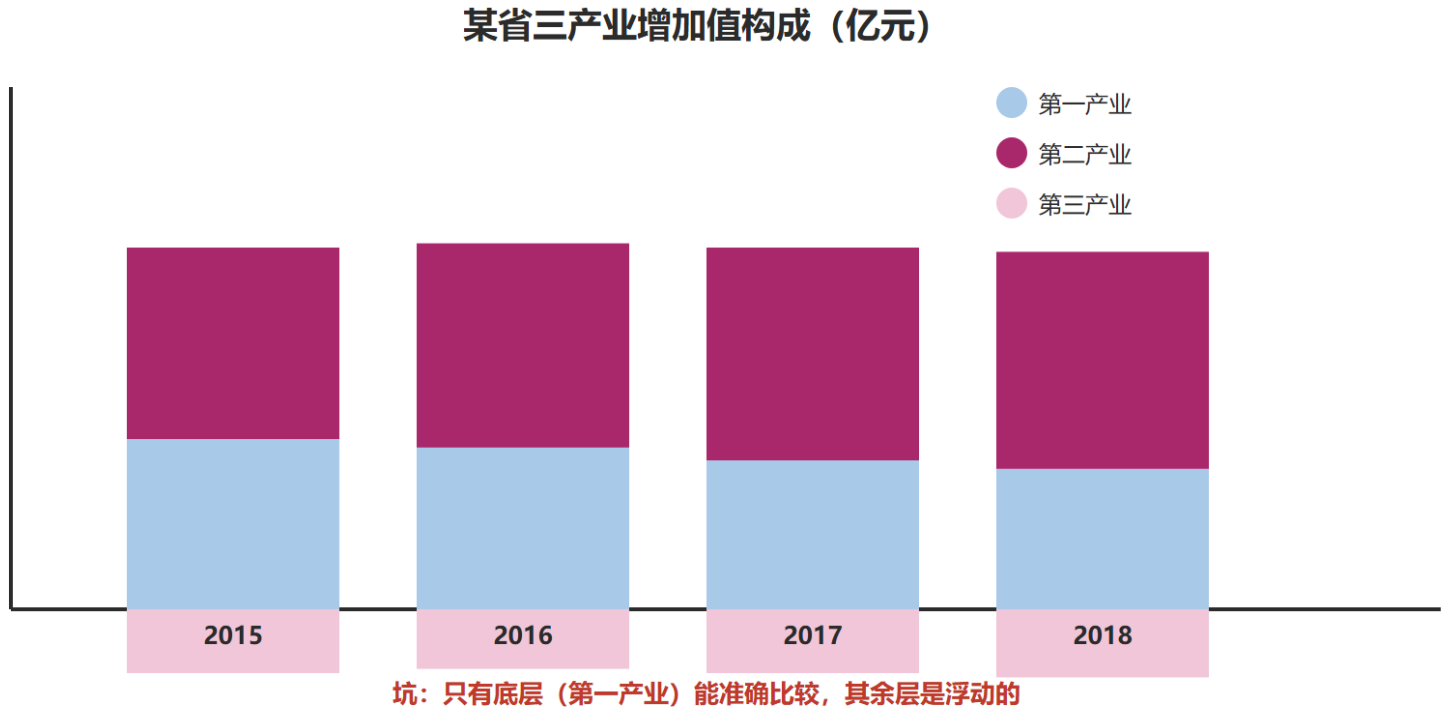
柱状图：比较大小的第一选择

- **三个细节：排序、柱顶标签、y 轴单位**
- 竖条柱状 / 横条条形，类别名长用条形
- 2017 年六省 GDP：一眼分出高下



堆叠图：看构成，但别堆太多层

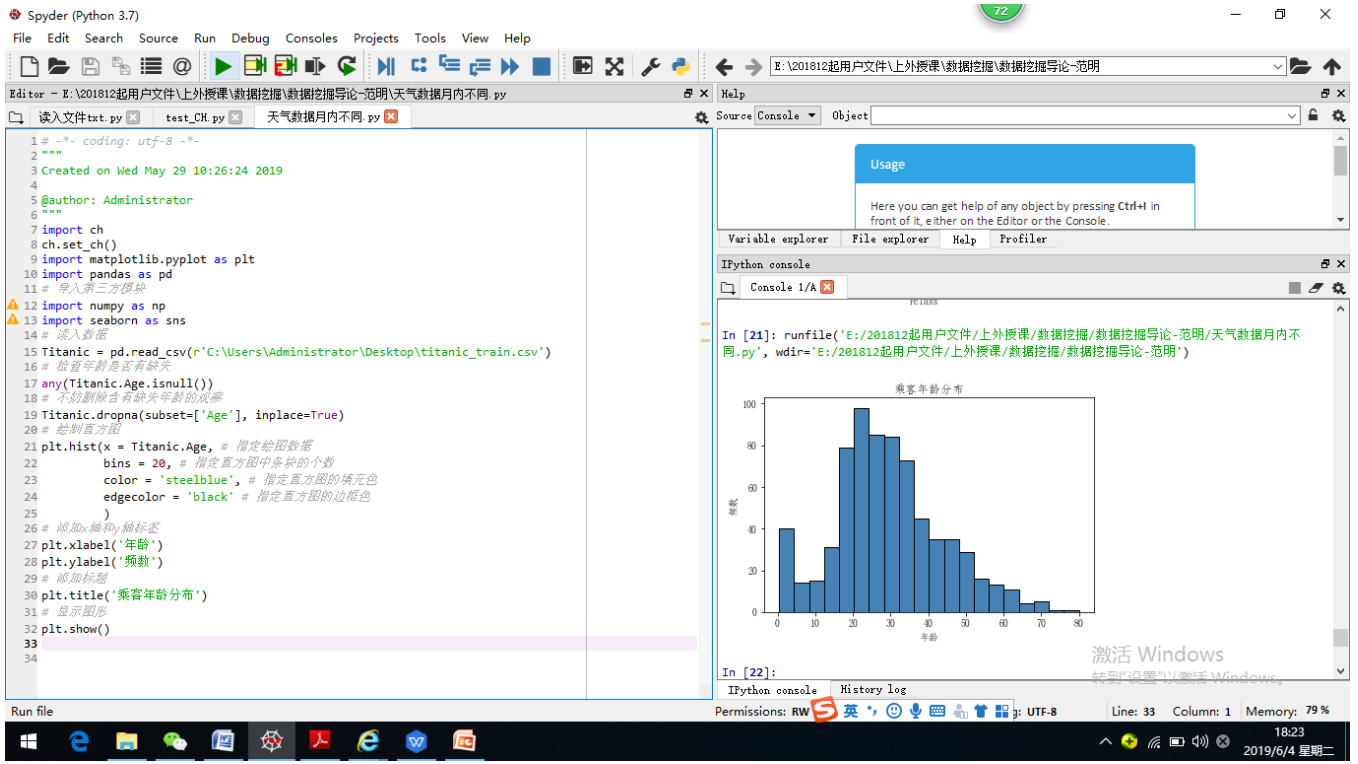
- **同时看总额 + 内部构成**
- 超过三层就很难看清
- 只有底层能准确比较，其余层是浮动的



坑：只有底层（第一产业）能准确比较，其余层是浮动的

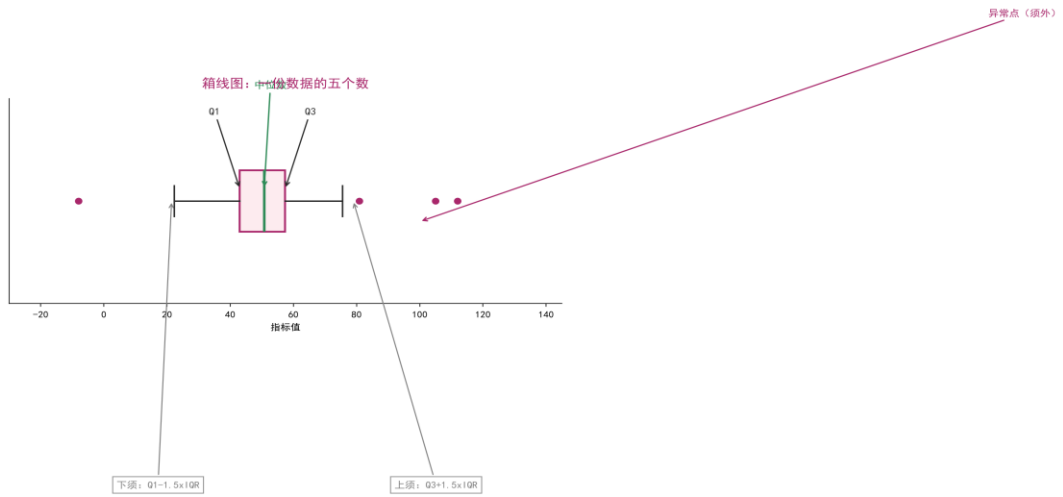
直方图：看分布长什么样

- 柱状图比类别，直方图看连续分布形状
- 关键参数 bins：太少看不出形状，太多全是噪声
- 现场改 bins 让学生看差别



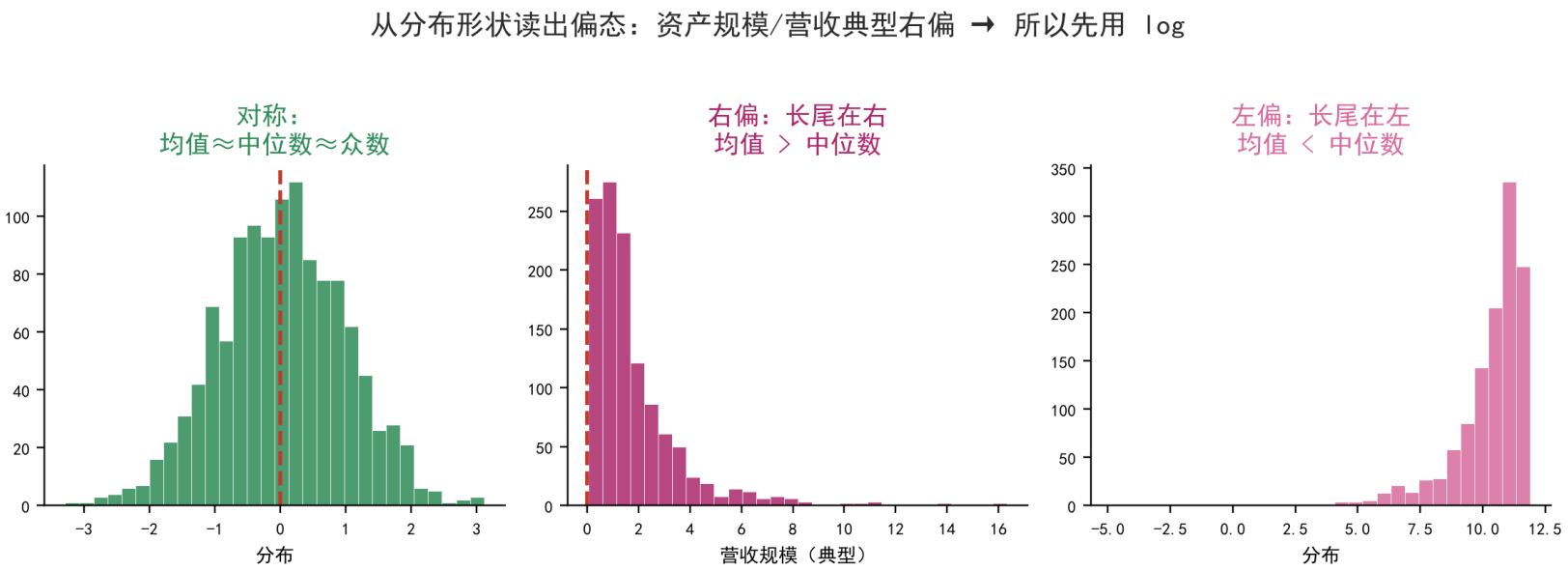
箱线图：一份数据的五个数

- **五个数：Q1 / 中位数 / Q3 / 上下须 / 异常点**
- 须 = Q1、Q3 ± 1.5×IQR
- 这就是离群点检测的具体画法



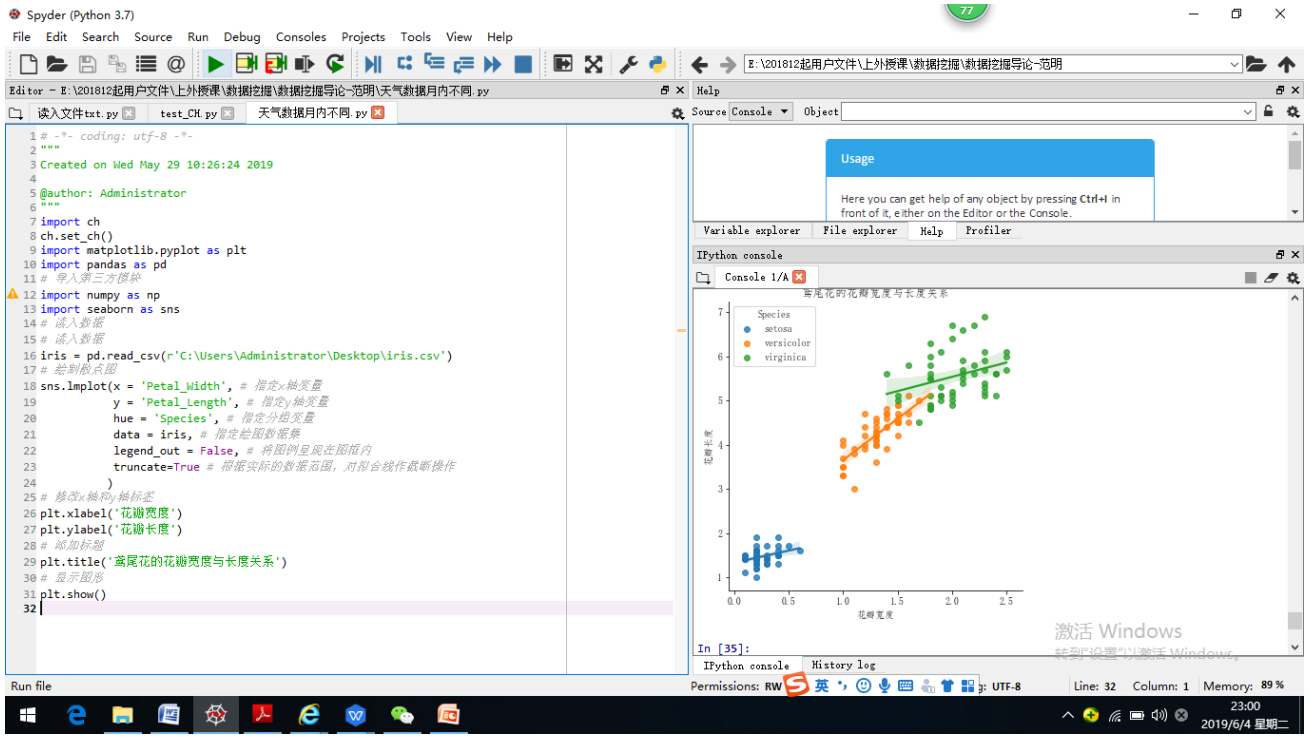
从箱线图形状读出偏态

- **对称：三者相等；右偏：众数 < 中位数 < 均值**
- 资产规模、营收典型右偏
- 所以前面才要 log 变换



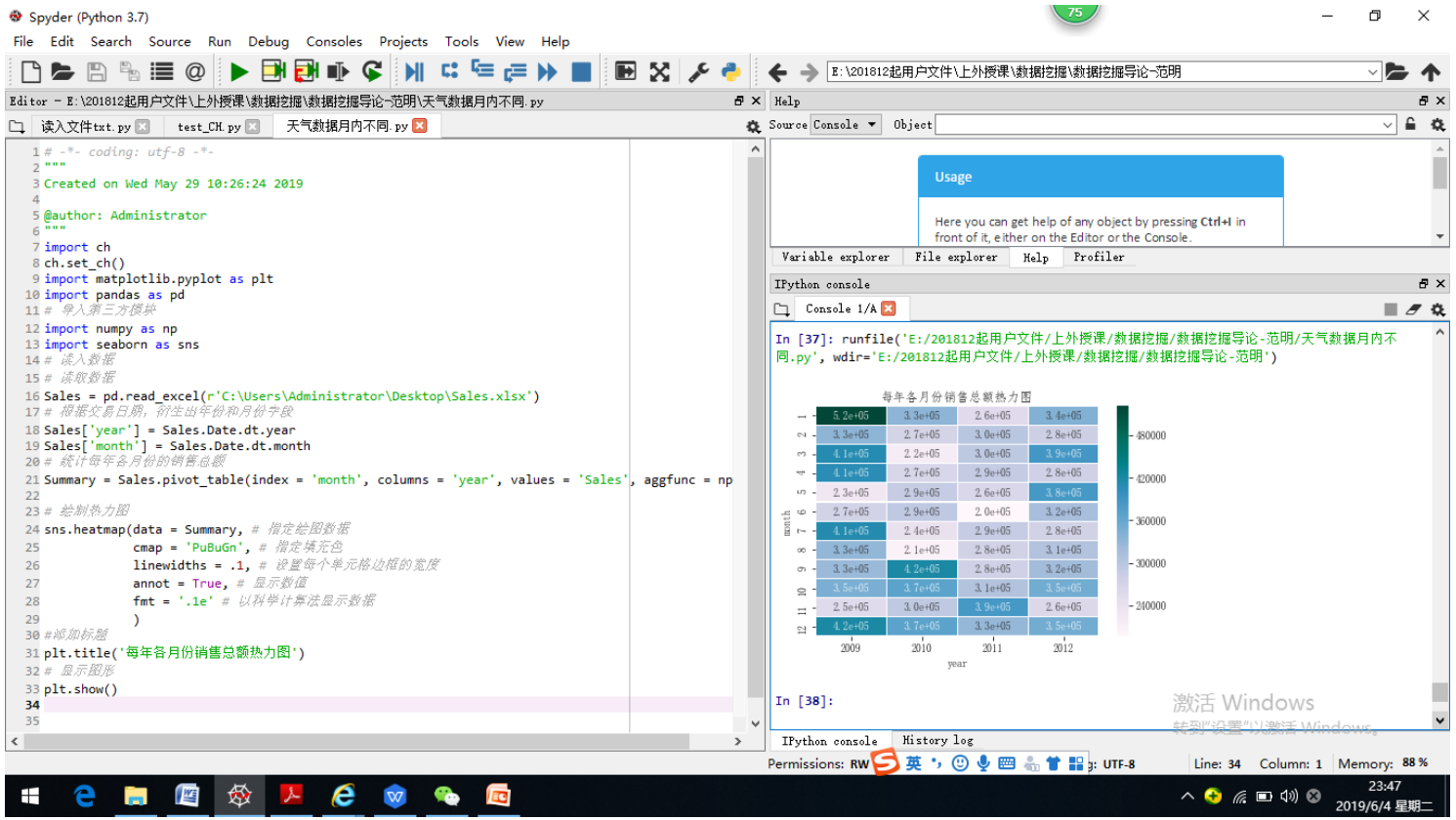
散点图：看两个变量的关系

- 两变量关系一目了然
- 用颜色装下第三个变量
- 公司聚成一簇 → 这两个指标能区分它们 → 建模直觉



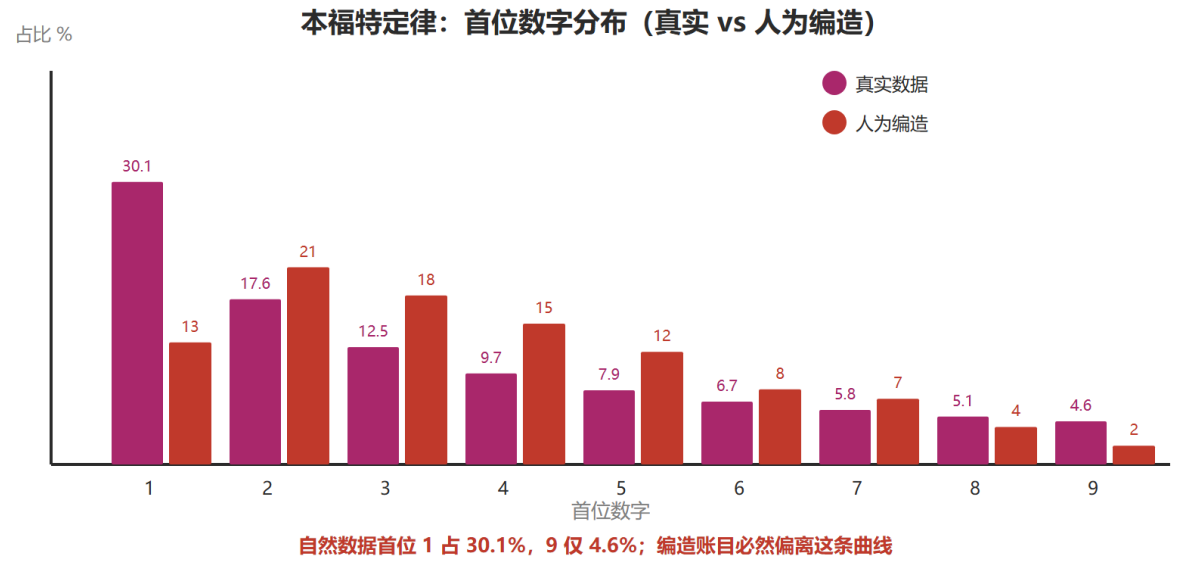
热力图：一眼看出哪里缺数据

- 公司 × 年份缺失热力图
- 一眼看出哪块数据是空的
- 先看见，再决定怎么填



本福特定律：可视化揪出财务造假

- **自然数据首位数字：1 占 30.1%，9 仅 4.6%**
- 人为编造的账，首位分布必然偏离
- 康美、瑞幸的质疑工具 —— 会计高光时刻



案例连接：拿合并后的财务字段首位数字画分布，对照理论曲线找偏离

Orange：拖拽式可视化，不写一行代码

Orange 是图形化数据分析工具 —— 把数据接进组件，连线即可出图（Day2 实操会用）

分布图 Distribution

看什么：单个变量的分布形状
用在哪：看偏态、离群、区间

箱线图 Box Plot

看什么：分组间的分布差异
用在哪：对比行业/年份中位数

小提琴图 Violin

看什么：分布形状 + 密度
用在哪：比箱线更细的分布细节

筛图 Sieve

看什么：实际 vs 期望频数
用在哪：特征与目标是否独立

马赛克图 Mosaic

看什么：多变量复合影响
用在哪：两三个分类变量交叉

线性投影 Linear Projection

看什么：高维映射到 2D
用在哪：找类别在低维的分离

Radviz

看什么：多维径向投影
用在哪：观察特征对类别的作用

FreeViz

看什么：自动优化坐标轴
用在哪：有监督地最大化类别分离

generate-chart：WorkBuddy 的绘图 skill

① 输入：数据文件
CSV / Excel
一句话需求

② 自动分析
数据类型/维度/相关性
→ 推荐最合适的图

③ 输出：交互仪表板
单个 HTML
悬停/缩放/平移

支持 10+ 种图：柱状图 • 折线图 • 饼图 • 散点图 • 直方图 • 箱线图 • 小提琴图 • 面积图 • 热力图 • 平行坐标图 • 旭日图

提示词：读 financial_data.csv，自动分析数据特征，
推荐最合适的图表组合，生成交互式 HTML 可视化仪表板

演示：一句话让 Agent 出图

- **给 WorkBuddy 的自然语言指令：**
- 读 financial_data.csv，画资产负债率直方图
- 按行业画箱线图，再输出公司×年份缺失热力图

03

数据获取

HTTP • 两条路线 • 三个故障

先看图：公开数据都藏在哪

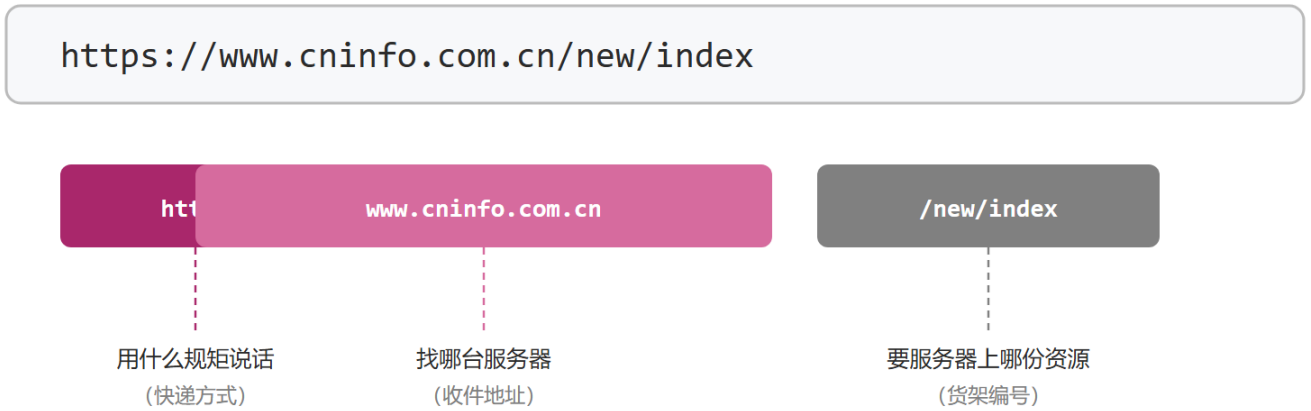
- **主战场：巨潮资讯网的年报 PDF**
- 取数据按披露日期，不是报表年度
- 其余来源点到为止



URL 三段：互联网的门牌号

- 协议 + 域名 + 路径，三段定位一个资源
- https 是加密版 http
- 问号后面是查询参数，爬虫最常用

一条 URL 的三个部分



换成 API 时，变的通常是第三段

HTTP 请求过程：从敲回车到看见页面

- **DNS 解析 → 建立连接 → 发请求 → 收响应 → 渲染**
- 爬虫只需模拟前几步，跳过渲染
- 这就是爬虫快的根本原因



超文本：浏览器看到的网页 = HTML 标签 + 文本 + 多媒体

什么是超文本（Hypertext）？

网页源代码 = 一系列 HTML 代码，由一系列**标签**组成

<img> 显示图片 • <p> 段落 • <a> 链接 • <table> 表格

怎么看网页源代码？

浏览器打开网页 → 鼠标右键 → "查看网页源代码"

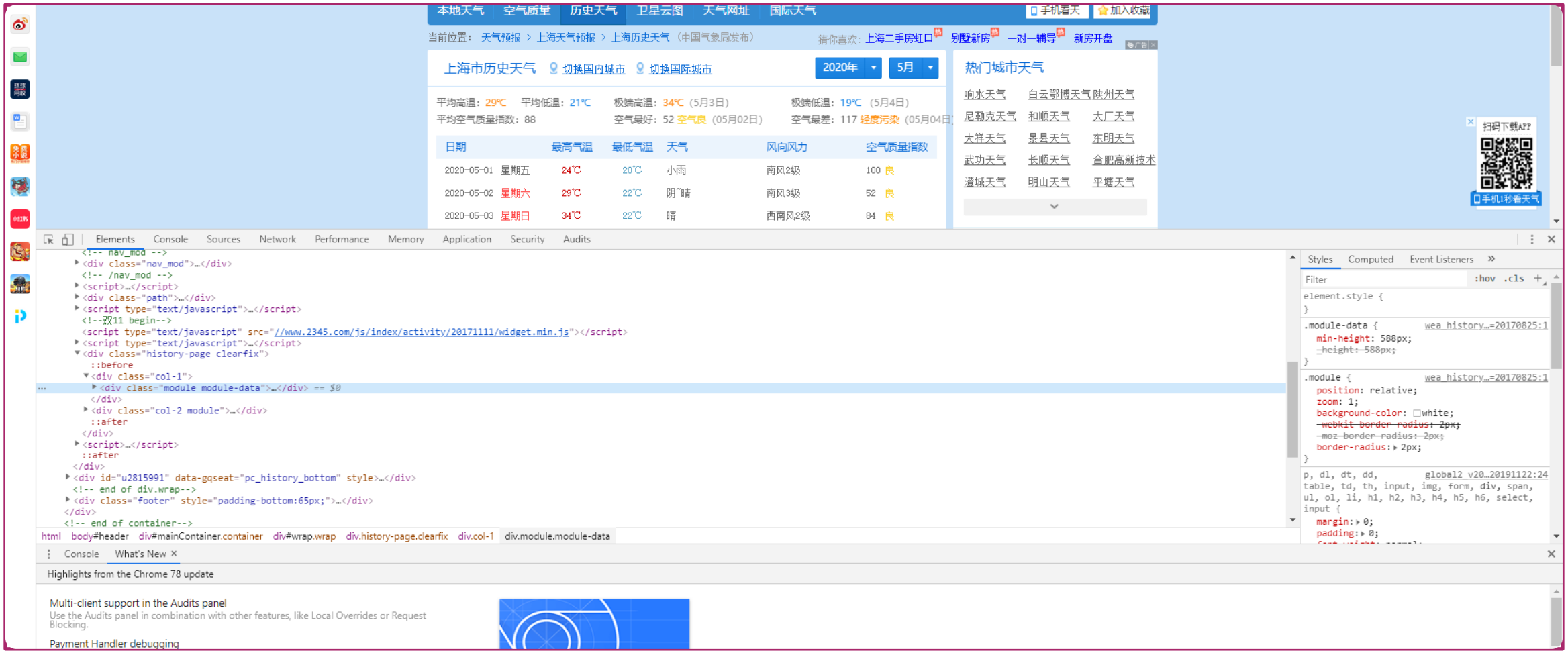
或按 **F12** 打开开发者工具

"审查元素" 能看到更多信息（标签位置、样式、计算后的 CSS）

```
<!-- 一个最简单的 HTML 文档 -->
<html>
  <head>
    <title>深圳大学 · 数据挖掘导论</title>
  </head>
  <body>
    <h1>欢迎学习爬虫</h1>
    <p>本章目标：理解 HTTP 协议</p>
  </body>
</html>
```

关键洞察：爬虫 = 用程序自动下载 HTML，再用正则或解析器提取想要的内容。**HTTP** 管"怎么传"，**HTML** 管"传的是什么"。

真实截图：F12 审查元素看网页结构（对应 P7）



HTTP：超文本传输协议

HTTP = HyperText Transfer Protocol

超文本传输协议：从网络传输超文本数据到本地浏览器的传送协议

由 W3C（万维网协会）和 IETF（Internet 工作小组）共同制定，目前广泛使用 HTTP/1.1

HTTP 报文结构

请求行 / 状态行

```
GET /index.html HTTP/1.1
HTTP/1.1 200 OK
```

请求头 / 响应头

Host / User-Agent / Accept / Content-Type ...

请求体 / 响应体

POST 表单数据 / HTML 源代码 / JSON

关键特性

- **请求-响应模式**：客户端发起请求，服务器返回响应
- **无状态**：每次请求独立，服务器不记得你之前来过
- **明文传输**：所有内容都能被中间人看到（这正是 HTTPS 要解决的问题）
- **基于 TCP**：先建立 TCP 连接，再发 HTTP 报文
- **默认端口 80**（HTTPS 是 443）

HTTPS：HTTP + SSL/TLS 安全层

HTTPS = HTTP over Secure Socket Layer

以**安全**为目标的 HTTP 通道 = HTTP 协议 + SSL/TLS 加密层

HTTP 协议栈

- HTTP 明文
- TCP
- IP / 网络

中间人能看到全部明文 ✗

HTTPS 协议栈

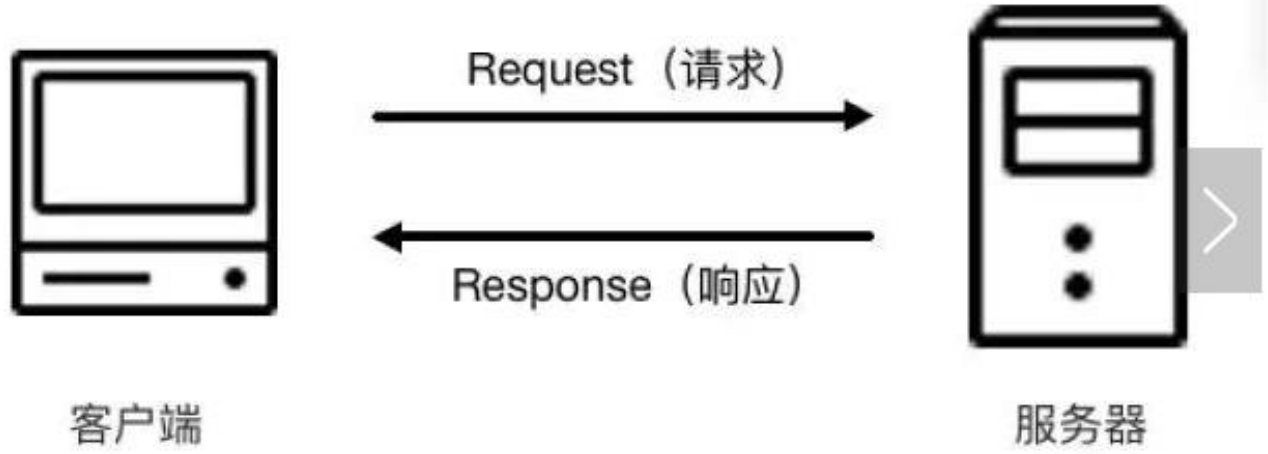
- HTTP
- SSL / TLS ← 加密层
- TCP
- IP / 网络

内容加密传输 ✓

识别 HTTPS：浏览器地址栏 🔒 锁头标志；点击可查看 CA 机构颁发的真实认证信息与安全签章。

真实截图：HTTP 请求过程模型（对应 P10）

个人图书馆 360doc.com 千万人在用的知识管理与分享平台 我的图书馆 搜文章 找馆友 留言交流



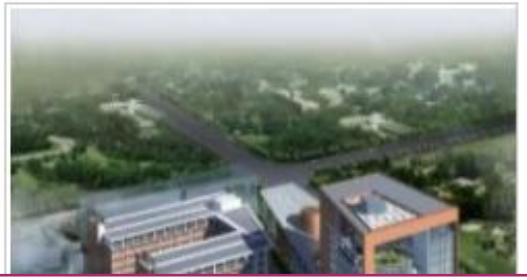
模型图

此处客户端即代表我们自己的 PC 或手机浏览器，服务器即要访问的网站所在的服务器。

为了更直观地地说明这个的过程，我们在这里用 Chrome 浏览器的开发者模式下的 Network 监听组件来做下演示，它可以显示访问当前请求网页时发生的所有网络请求和响应。



全国MBA排名 查看详情 关闭



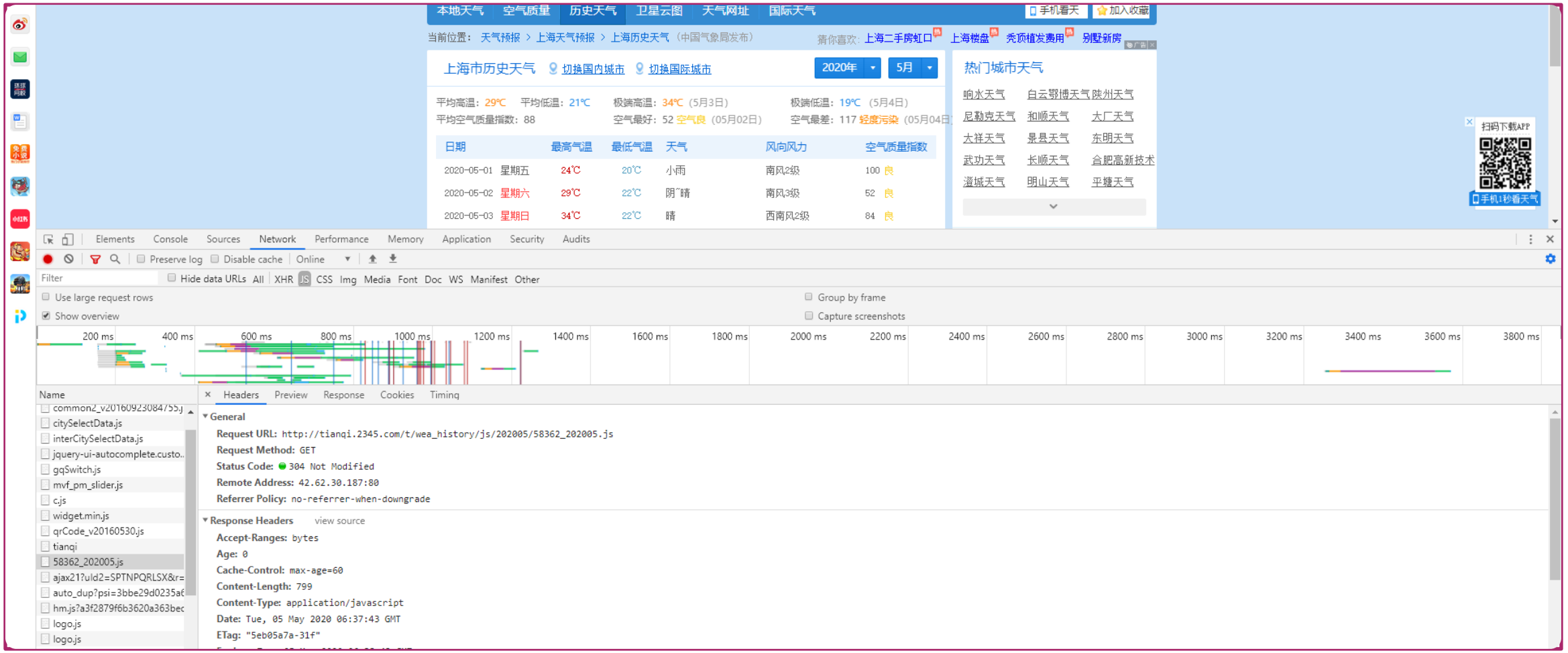
HTTP 请求过程：从输入 URL 到看见页面

1	2	3	4	5	6
DNS 解析	TCP 连接	发送 Request	服务器处理	返回 Response	浏览器渲染

关键洞察

- 爬虫要做的 = **模拟第 1-3 步**（跳过浏览器渲染）
- 第 5 步 Response Body 就是爬虫要解析的**目标数据**
- `requests.get(url)` 一行代码 = 自动完成 1+2+3+5 四步
- 浏览器 F12 开发者工具的 Network 面板能**逐条观察**每次请求的细节
- 爬虫的**headers** = 模仿浏览器的"身份证"

真实截图：Network 面板观察 HTTP 请求（对应 P11）



用浏览器实际观察一次请求

操作三步走

1. 浏览器输入 URL，回车
2. 按 F12 → Network 面板
3. 点击第一条请求，看 Request / Response Headers

Network 面板 = 浏览器和服务器之间的"对账单"，每次 HTTP 请求都被记下来

爬虫开发时第一件事：先打开 Network 看看正常浏览器都发了什么字段

— Network 面板 ————————————————————————

```
GET  https://github.com/favicon.ico
Status: 200 OK · Size: 1.2 KB · Time: 87 ms
```

▼ Request Headers

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: image/webp,*/*;q=0.8
Referer: https://github.com/
Cookie: logged_in=no; ...
```

▼ Response Headers

```
Server: GitHub.com
Content-Type: image/x-icon
Cache-Control: max-age=86400
```

Request：客户端对服务器说

Request = 请求，由客户端向服务端发出

一个 Request 包含 **4 个部分**： Method • URL • Headers • Body

1
Method
请求方法

`GET / POST`

告诉服务器"做什么动作"

2
URL
请求地址

`/index.html`

告诉服务器"做什么资源"

3
Headers
请求头

`User-Agent...`

告诉服务器"我是谁/我要啥格式"

4
Body
请求体

表单数据

告诉服务器"我要提交什么"

Request Method：8 种动词，最常用的是 GET 与 POST

GET

请求指定页面信息

- 参数在 URL 中可见
- 数据最多 1024 字节
- 爬虫**默认**使用

POST

向指定资源提交数据

- 数据在 Request Body
- 数据大小无限制
- 模拟登录、提交表单用

8 种 HTTP Method

GET	请求页面信息，返回实体主体
HEAD	类似 GET，响应中无具体内容，只有报头
POST	向指定资源提交数据进行处理请求
PUT	从客户端向服务器传送的数据取代指定文档
DELETE	请求服务器删除指定的页面
OPTIONS	允许客户端查看服务器的性能
CONNECT	HTTP/1.1 预留给代理服务器的管道方式
TRACE	回显服务器收到的请求，用于测试或诊断

Request URL 与 Request Headers 概览

Request URL

请求的网址（统一资源定位符）

```
https://github.com/favicon.ico
```

https://协议

github.com域名

/favicon.ico路径

URL 唯一确定我们想请求的资源

Request Headers

请求头：服务器要使用的附加信息

Accept：客户端可接受的响应类型

Accept-Language：客户端可接受的语言类型

Accept-Encoding：客户端可接受的内容编码

Host：请求资源的主机 IP 和端口号

Cookie：维持当前访问会话

Referer：请求从哪个页面发出

User-Agent：客户端 OS / 浏览器信息

Content-Type：请求体的媒体类型

Cookie / Referer / User-Agent：爬虫最该关注的三个

Cookie

维持登录会话

网站为了辨别用户进行 Session 跟踪而存储在用户本地的数据

示例： 登录某网站后，每次请求都会带上 Cookie，服务器就知道"你是谁"

爬虫要点： 爬虫需要登录后才能访问的页面 → 必须带上 Cookie

Referer

请求来源页面

标识请求是从哪个页面发出的

示例： 从百度点进 GitHub，Referer = https://baidu.com/

爬虫要点： 防盗链 / 来源统计：很多图片 / 资源拒绝直接 URL 访问

User-Agent

客户端身份

客户端 OS、浏览器及版本信息

示例： Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36

爬虫要点： 不加 UA 很容易被识别为爬虫 → 必须伪装成浏览器

Request Body：POST 的内容装在这里

Request Body

请求体 = POST 请求中的 Form Data；GET 请求则为空

application/x-www-form-urlencoded

Form Data 形式提交

键值对用 & 分隔

爬虫要根据 Content-Type 选用正确的解析方式

multipart/form-data

表单文件上传

用于上传图片/文件

爬虫要根据 Content-Type 选用正确的解析方式

application/json

序列化 JSON 数据

API 接口最常见

爬虫要根据 Content-Type 选用正确的解析方式

text/xml

XML 数据

老式 SOAP 接口

爬虫要根据 Content-Type 选用正确的解析方式

Response Status Code：200/404/500 的含义

状态码 = 服务器的"回应表情"，5 类共约 60 个常用值

1xx	信息响应	Information	100 Continue
2xx	成功	Success	200 OK · 201 Created · 204 No Content
3xx	重定向	Redirection	301 Moved · 302 Found · 304 Not Modified
4xx	客户端错误	Client Error	400 Bad Request · 401 Unauthorized · 404 Not Found · 403 Forbidden
5xx	服务器错误	Server Error	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

真实截图：HTTP 状态码详细说明

个人图书馆 360doc.com 千万人在用的知识管理与分享平台 我的图书馆 搜文章 找馆友 留言交流

Response，即响应，由服务端返回给客户端。Response 可以划分为三部分，Response Status Code、Response Headers、Response Body。

Response Status Code

响应状态码，此状态码表示了服务器的响应状态，如 200 则代表服务器正常响应，404 则代表页面未找到，500 则代表服务器内部发生错误。在爬虫中，我们可以根据状态码来判断服务器响应状态，如判断状态码为 200，则证明成功返回数据，再进行进一步的处理，否则直接忽略。

下面用表格列出了常见的错误代码及错误原因：

状态码说明详情

100继续请求者应当继续提出请求。服务器已收到请求的一部分，正在等待其余部分。

101切换协议请求者已要求服务器切换协议，服务器已确认并准备切换。

200成功服务器已成功处理了请求。

201已创建请求成功并且服务器创建了新的资源。

202已接受服务器已接受请求，但尚未处理。

分享 更多



关闭



我要转藏

Response Headers：服务器告诉浏览器的元信息

响应头包含：Content-Type / Server / Set-Cookie 等

Date：Response 产生的时间

Last-Modified：指定资源的最后修改时间

Content-Encoding：指定 Response 内容的编码

Server：服务器信息（名称、版本号）

Content-Type：返回的数据类型（text/html / image/jpeg / application/json）

Set-Cookie：告诉浏览器需要将此内容放在 cookie 中

Expires：Response 的过期时间，控制缓存加载

Response Body：爬虫真正要解析的东西

Response Body = 响应体

请求一个网页 → Body 是 HTML 代码；请求一张图片 → Body 是图片二进制数据

HTML 文档

```
Content-Type
text/html
```

网页源代码，用 re / BeautifulSoup 解析

← 爬虫要拿的目标

JSON 数据

```
Content-Type
application/json
```

API 接口最常用，requests.json() 直接拿字典

← 爬虫要拿的目标

二进制文件

```
Content-Type
image/jpeg, video/mp4
```

图片/视频/PDF，需 wb 写入文件

← 爬虫要拿的目标

2345 天气爬虫：5 步路线图

抓取 2345 历史天气（2011-2017）共 7 年数据，生成 CSV 文件

1 导入包

2 构造请求头

3 生成 URL 列表

4 正则匹配数据

5 合并 + 导出 CSV

关键技术栈

requests
发送 URL 请求

time
时间停顿

random
随机数生成

pandas
数据合并与导出

re
正则表达式

Step 1：导入第三方包

```
# 导入第三方包
import requests     # 用于 Url 的请求和数据获取
import time         # 用于时间的停顿
import random       # 用于随机数的生成
import pandas as pd    # 用于数据的导出
import re           # 用于正则表达式的使用
```

5 个库的角色

- **requests** —— 与服务器对话
- **time.sleep** —— 礼貌停顿
- **random** —— 停顿时长随机
- **pandas** —— 数据清洗 + 导出 CSV
- **re** —— 从响应文本里抠数据

安装方法：`pip install requests pandas`

Step 2：构造请求头（伪装成浏览器）

```
# 生成请求头
headers = {
  'Accept': '*/*',
  'Accept-Encoding': 'gzip, deflate',
  'Accept-Language': 'zh-CN,zh;q=0.9',
  'Connection': 'keep-alive',
  'Cookie': 'widget_dz_id=54511; ...',
  'User-Agent': 'Mozilla/5.0 (Windows NT 6.1; WOW64) Chrome/63.0...'
}
```

为什么需要请求头？

- **User-Agent**：告诉服务器"我是 Chrome 浏览器"（不带 UA 容易被识别为爬虫）
- **Cookie**：维持会话状态，告诉服务器"我之前来过"
- 真实爬虫第一步：**从浏览器 F12 Network 复制真实请求头**

Step 3：生成所有需要抓取的 URL（双重循环）

```
urls = []
for year in range(2011, 2018):
  for month in range(1, 13):
    if year <= 2016:
      urls.append('http://tianqi.2345.com/.../58362_
      %s%s.js' %(year, month))
    else:
      if month < 10:
        urls.append('.../58362_%s0%s.js' %(year, month))
      else:
        urls.append('.../58362_%s%s.js' %(year, month))
```

关键洞察

- 7 年 × 12 月 = 84 个 URL
- 2011–2016 与 2017 的 URL 模式略有不同

→ 必须分支处理

- 2017 年的 1–9 月，月份前面要补 0

真实项目常见难点

URL 规则随时间变化，必须先观察再写代码

Step 4：循环 + 正则匹配（核心数据提取）

```
info = []
for url in urls:
  seconds = random.randint(3, 6)
  response = requests.get(url, headers=headers).text
  ymd    = re.findall(r"ymd:'(.*?)',", response)
  high   = re.findall(r"bWendu:'(.*?)℃',", response)
  low    = re.findall(r"yWendu:'(.*?)℃',", response)
  tianqi= re.findall(r"tianqi:'(.*?)',", response)
  fengxiang = re.findall(...)
  aqi   = re.findall(r"aqi:'(.*?)',", response)
  ...
```

这一步发生了什么？

- **每个 URL 走一遍**：发请求 → 拿回 HTML
- **8 个 re.findall**：抠出日期/最高温/最低温/天气/风向/风力/AQI/空气质量
- **time.sleep(random)**：每次循环随机停 3-6 秒

为什么停顿？

连续高频请求会被服务器识别为爬虫并封禁 IP；礼貌爬虫每次都停几秒

Step 5：数据合并与导出 CSV

```
# 将存储的所有天气数据进行合并，生成数据表
weather = pd.concat(info)

# 数据导出
weather.to_csv('weather.csv', index=False)
```

pd.concat(info)

把 84 个月份的小 DataFrame 拼成一张大表

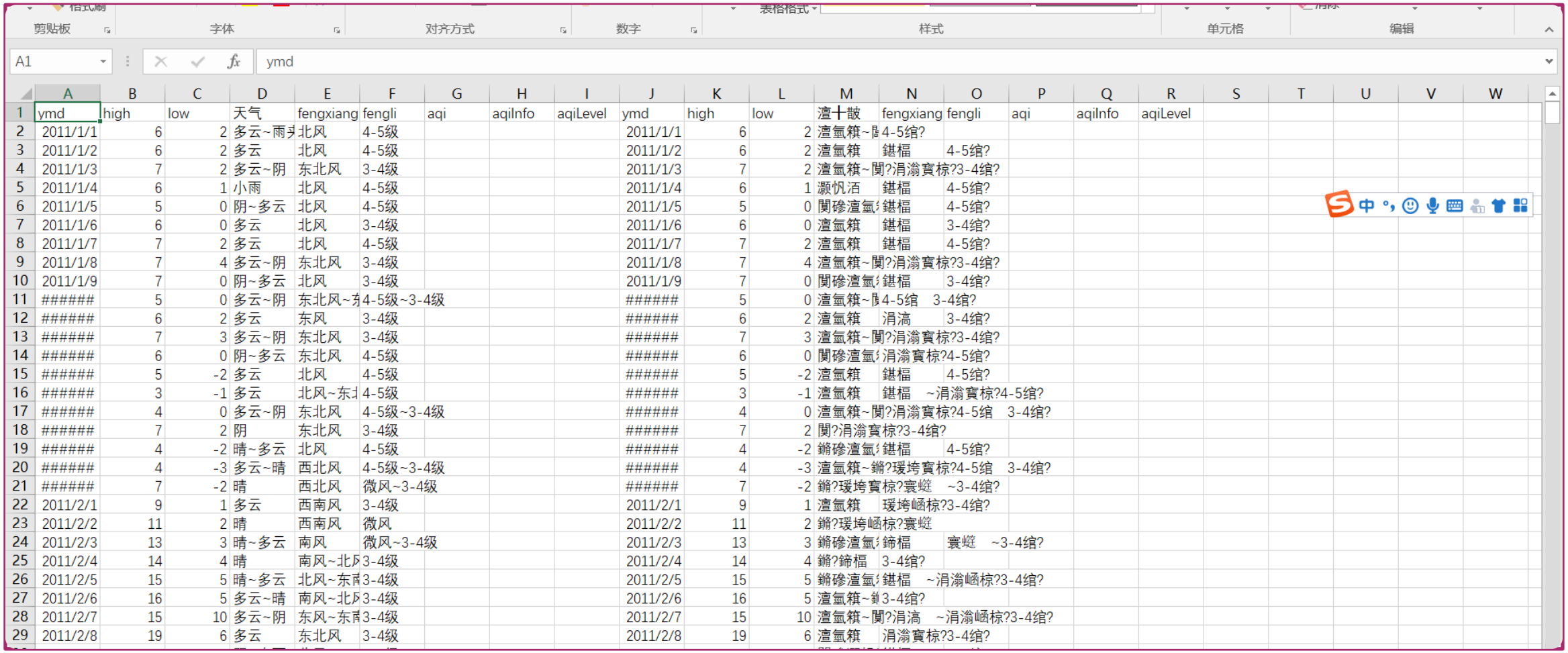
- info 是一个列表，每一项是一个月的 df
- concat 按行拼接，列名自动对齐
- 最终形状 ≈ (7×365, 9)

to_csv(index=False)

导出到本地 CSV 文件

- index=False = 不把 pandas 的行索引写进 CSV
- 输出文件可直接用 Excel / pandas / Orange 打开
- 至此一个完整"互联网数据 → 本地数据"的链路打通

真实截图：weather.csv 在 Excel 中的运行结果（对应 P27）



剪贴板 字体 对齐方式 数字 样式 单元格 编辑

A1 ymd

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	ymd	high	low	天气	fengxiang	fengli	aqi	aqiInfo	aqiLevel	ymd	high	low	澶╂皵	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2	2011/1/1	6	2	多云~雨夹	北风	4-5级				2011/1/1	6	2	澶氫簮~	4-5绾?				
3	2011/1/2	6	2	多云	北风	4-5级				2011/1/2	6	2	澶氫簮	鍖楅	4-5绾?			
4	2011/1/3	7	2	多云~阴	东北风	3-4级				2011/1/3	7	2	澶氫簮~	闐?涓滃寘椋?3-4绾?				
5	2011/1/4	6	1	小雨	北风	4-5级				2011/1/4	6	1	灏忚洦	鍖楅	4-5绾?			
6	2011/1/5	5	0	阴~多云	北风	4-5级				2011/1/5	5	0	闐毯澶氫	鍖楅	4-5绾?			
7	2011/1/6	6	0	多云	北风	3-4级				2011/1/6	6	0	澶氫簮	鍖楅	3-4绾?			
8	2011/1/7	7	2	多云	北风	4-5级				2011/1/7	7	2	澶氫簮	鍖楅	4-5绾?			
9	2011/1/8	7	4	多云~阴	东北风	3-4级				2011/1/8	7	4	澶氫簮~	闐?涓滃寘椋?3-4绾?				
10	2011/1/9	7	0	阴~多云	北风	3-4级				2011/1/9	7	0	闐毯澶氫	鍖楅	3-4绾?			
11	######	5	0	多云~阴	东北风~东	4-5级~3-4级				######	5	0	澶氫簮~	4-5绾	3-4绾?			
12	######	6	2	多云	东风	3-4级				######	6	2	澶氫簮	涓滆	3-4绾?			
13	######	7	3	多云~阴	东北风	3-4级				######	7	3	澶氫簮~	闐?涓滃寘椋?3-4绾?				
14	######	6	0	阴~多云	东北风	4-5级				######	6	0	闐毯澶氫	涓滃寘椋?4-5绾?				
15	######	5	-2	多云	北风	4-5级				######	5	-2	澶氫簮	鍖楅	4-5绾?			
16	######	3	-1	多云	北风~东北	4-5级				######	3	-1	澶氫簮	鍖楅	~涓滃寘椋?4-5绾?			
17	######	4	0	多云~阴	东北风	4-5级~3-4级				######	4	0	澶氫簮~	闐?涓滃寘椋?4-5绾	3-4绾?			
18	######	7	2	阴	东北风	3-4级				######	7	2	闐?涓滃寘椋?3-4绾?					
19	######	4	-2	晴~多云	北风	4-5级				######	4	-2	鏘毯澶氫	鍖楅	4-5绾?			
20	######	4	-3	多云~晴	西北风	4-5级~3-4级				######	4	-3	澶氫簮~	鏘?瑗垮寘椋?4-5绾	3-4绾?			
21	######	7	-2	晴	西北风	微风~3-4级				######	7	-2	鏘?瑗垮寘椋?褰嶇	~3-4绾?				
22	2011/2/1	9	1	多云	西南风	3-4级				2011/2/1	9	1	澶氫簮	瑗垮崳椋?3-4绾?				
23	2011/2/2	11	2	晴	西南风	微风				2011/2/2	11	2	鏘?瑗垮崳椋?褰嶇					
24	2011/2/3	13	3	晴~多云	南风	微风~3-4级				2011/2/3	13	3	鏘毯澶氫	鍗楅	褰嶇	~3-4绾?		
25	2011/2/4	14	4	晴	南风~北风	3-4级				2011/2/4	14	4	鏘?鍗楅	3-4绾?				
26	2011/2/5	15	5	晴~多云	北风~东南	3-4级				2011/2/5	15	5	鏘毯澶氫	鍖楅	~涓滃崳椋?3-4绾?			
27	2011/2/6	16	5	多云~晴	南风~北风	3-4级				2011/2/6	16	5	澶氫簮~	鏘3-4绾?				
28	2011/2/7	15	10	多云~阴	东风~东南	3-4级				2011/2/7	15	10	澶氫簮~	闐?涓滆	~涓滃崳椋?3-4绾?			
29	2011/2/8	19	6	多云	东北风	3-4级				2011/2/8	19	6	澶氫簮	涓滃寘椋?3-4绾?				

爬虫运行结果：weather.csv

输出文件：**weather.csv**（约 2500 行 × 9 列）

ymd	high	low	tianqi	fengxiang	fengli	aqi	aqiInfo	aqiLevel
2017-01-01	8℃	-3℃	晴	东北风	3-4级	62	良	2
2017-01-02	9℃	-2℃	多云	东南风	2-3级	78	良	2
2017-01-03	7℃	-1℃	阴	北风	3-4级	85	良	2
2017-01-04	6℃	0℃	小雨	东北风	2级	110	轻度污染	3
2017-01-05	5℃	-2℃	雨夹雪	北风	3-4级	95	良	2
2017-01-06	7℃	-4℃	晴	西北风	4-5级	55	良	2
2017-01-07	9℃	-1℃	多云	西风	2级	70	良	2
2017-01-08	10℃	0℃	晴	南风	2-3级	88	良	2

爬虫的边界：合规、伦理与法律

技术能力 ≠ 行动许可：爬虫能爬到 ≠ 应该爬 ≠ 法律允许爬

robots.txt

网站公布的"爬虫规则"

每个网站根目录下都有 robots.txt，声明哪些路径允许/禁止爬取

```
User-agent: *
Disallow: /admin/
Allow: /public/
```

数据隐私

个人信息 ≠ 可爬数据

《个人信息保护法》《网络安全法》严格约束个人数据的采集与使用

禁止爬取：手机号 / 身份证 / 通讯录 / 行踪轨迹 / 健康生理信息

版权与商业

数据不是"白拿的"

爬取的数据如用于商业目的，可能违反《反不正当竞争法》或网站 ToS

案例：百度诉奇虎违反 robots 案（2013）

反爬机制与应对策略

反爬机制	触发场景	应对策略
IP 频率限制	同一 IP 短时间内请求过多	降低频率 / 使用代理池
User-Agent 检测	请求头 UA 为空或缺失	复制浏览器的真实 UA
Cookie / 登录验证	需要登录才能访问的页面	requests.Session() 维持 Cookie
验证码	高频访问后弹图形验证码	接入打码平台 / 减少频率
蜜罐陷阱	隐藏链接诱导爬虫	只爬可见链接，避免 follow 隐藏 URL
JS 动态渲染	页面用 JS 异步加载数据	Selenium / Playwright 模拟浏览器

进阶工具：Requests / BeautifulSoup / Scrapy

Requests

入门级 HTTP 客户端

最常用的 HTTP 库，自动处理 Cookie / 连接池 / 编码

优点：

✓ API 极简

✓ 文档清晰

✓ 社区活跃

局限：

✗ 不能解析 HTML（需配 BS4）

✗ 无并发支持

BeautifulSoup

HTML / XML 解析器

把混乱的 HTML 解析成可遍历的对象树，按标签 / 属性取数据

优点：

✓ 容错强

✓ 支持 CSS 选择器

✓ 学习成本低

局限：

✗ 速度慢（lxml 解析器可加速）

✗ 需配合 requests 使用

Scrapy

工业级爬虫框架

一个完整项目框架：请求调度 / 并发 / 中间件 / 数据管道 / 持久化

优点：

✓ 高并发（异步）

✓ 完整的工程化方案

✓ 支持分布式

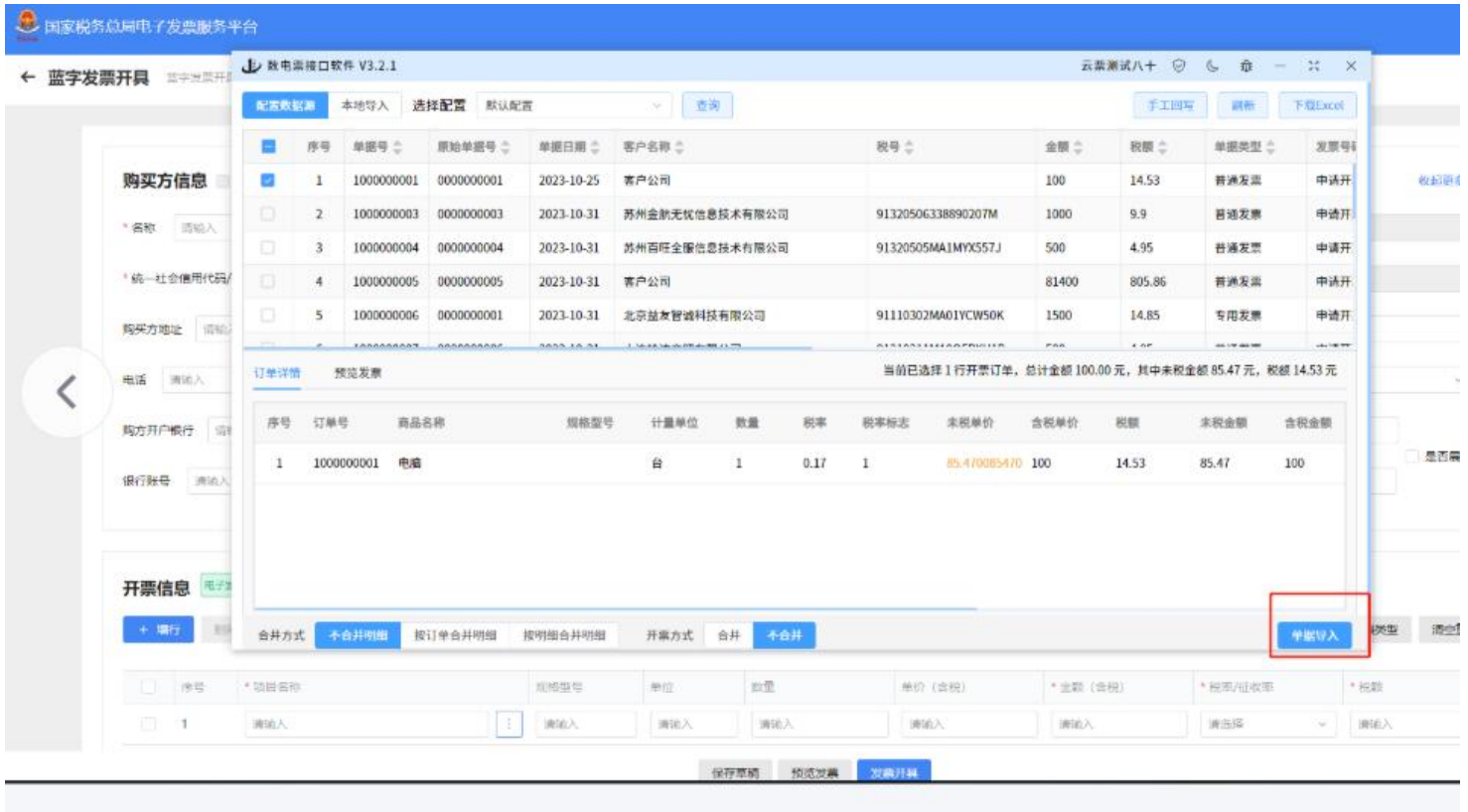
局限：

✗ 学习曲线陡

✗ 配置复杂

两条取数路线：八爪鱼与 Agent

- **八爪鱼解决「拿得到」**
- Agent 解决「说得清它从哪来」
- 规则文件不变，换工具只是换入口



Agent 代采集：人是审查者

- **把上午的规则文件丢给它**
- 严格按字段口径与输出标准执行
- 每步人确认 —— Agent 干活，你把关



用 WorkBuddy 抓巨潮财报：browser-skill 实战

提示词

帮我从巨潮资讯网下载 贵州茅台(600519) 的 2024 年年度报告 PDF

打开 www.cninfo.com.cn，用搜索框找到这只股票

进入定期报告 → 年度报告 → 下载 2024 年报 PDF 到 ./数据/年报/

下载完成后，用 csv 表格整理出：净利润、营业收入、资产负债率



browser-skill：WorkBuddy 的浏览器之手

一个 Skill：让 WorkBuddy 能操作你的真实浏览器

自动化操作：打开网页、搜索、点击、下载、填表

支持提前录制操作：你点一遍，它记住流程复现

需要先在浏览器装配套插件，才能被驱动

安装三步：

1. WorkBuddy 里启用 browser-skill
2. 浏览器安装配套插件/扩展
3. 授权连接，之后 AI 就能驱动浏览器

适合：需要登录、点按钮、下载文件的站点

三种爬虫方式，怎么选

直接爬 API：requests 一行，最干净最快

无头浏览器：Playwright/Selenium，能过 JS 渲染

MCP + browser-skill：真实浏览器 + AI 决策

本项目：能走 API 就走 API，要登录就用 browser-skill



直接爬 API：浏览器打开 URL 就是请求

有的网站直接把数据开放成 API —— 最省事

例：天气 API，浏览器直接打开 URL 就能看到 JSON

爬虫只需 requests.get(该URL) 拿回数据

演示：打开天气 API URL，看返回的 JSON 长什么样

浏览器地址栏：

https://api.weather.com/now?city=shenzhen

返回 JSON：

```
{ "city": "深圳", "temp": 26, "humidity": 65,
  "wind": "东南风3级", "aqi": 42 }
```

爬虫版 = requests.get(同款URL).json()

三个必然会遇到的故障

- **接下来三个故障，是刻意留下来的**
- 每个都会看到报错，然后当场修
- 目的是让大家知道出问题时长什么样，该怎么找原因
- 提示 Agent 时，把报错信息整段贴给它

故障一：页面是空的

- 内容是 JS 渲染出来的，直接请求 HTML 拿不到
- 解法一：找它背后的 API
- 解法二：无头浏览器

这不是你代码写错了，是数据还没到

requests 拿到的源码

```
<html>
<head>...</head>
<body>
<div id='app'></div>
</body>
</html>
```

静态抓取看到的是骨架　　表格呢？没有

浏览器渲染后的页面

公司代码	资产负债率	营业收入
600519	0.42	1508 亿
000651	0.61	2050 亿
300750	0.55	3620 亿
002027	0.33	418 亿
600036	0.91	3373 亿

真正的数据由 JS 后加载

解法：找它背后的 API（推荐），或用无头浏览器

故障二：PDF 表格错位

- **合并单元格导致列对不齐**
- 换解析库，或按坐标提取
- 必要时人工抽检核对

PDF 是给人看的，不是给机器读的

财报里的表格（合并单元格）

项目	2024	2023
资产	流动资产	
	非流动资产	
	合计	

一个格子跨三行 → 解析后必然错位

解析出来的 CSV

```
资产,,
流动资产,1200,1100
非流动资产,,,
,2300,2100
```

错位的典型症状

· 列对不齐，值跑到别的字段
· 合并行撕裂成空白
· 数字错位导致口径全错

正确处置顺序

① 优先找 XBRL / 结构化接口
② 换更强的 PDF 解析库
③ 按坐标提取
④ 必要时人工抽检核对

故障三：编码炸了

- **UTF-8 与 GBK 那段讲的现场版**
- 读文件永远显式写明编码
- 别依赖默认值

遇到乱码先问编码，不要先怀疑数据

报错

```
UnicodeDecodeError: 'gbk' codec can't decode byte 0x8b
in position 12: illegal multibyte sequence
```



```
pd.read_csv(..., encoding='utf-8')
```

修复后

```
证券代码 证券简称 资产负债率
0 600519 贵州茅台 0.42
1 000651 格力电器 0.61
2 300750 宁德时代 0.55
```



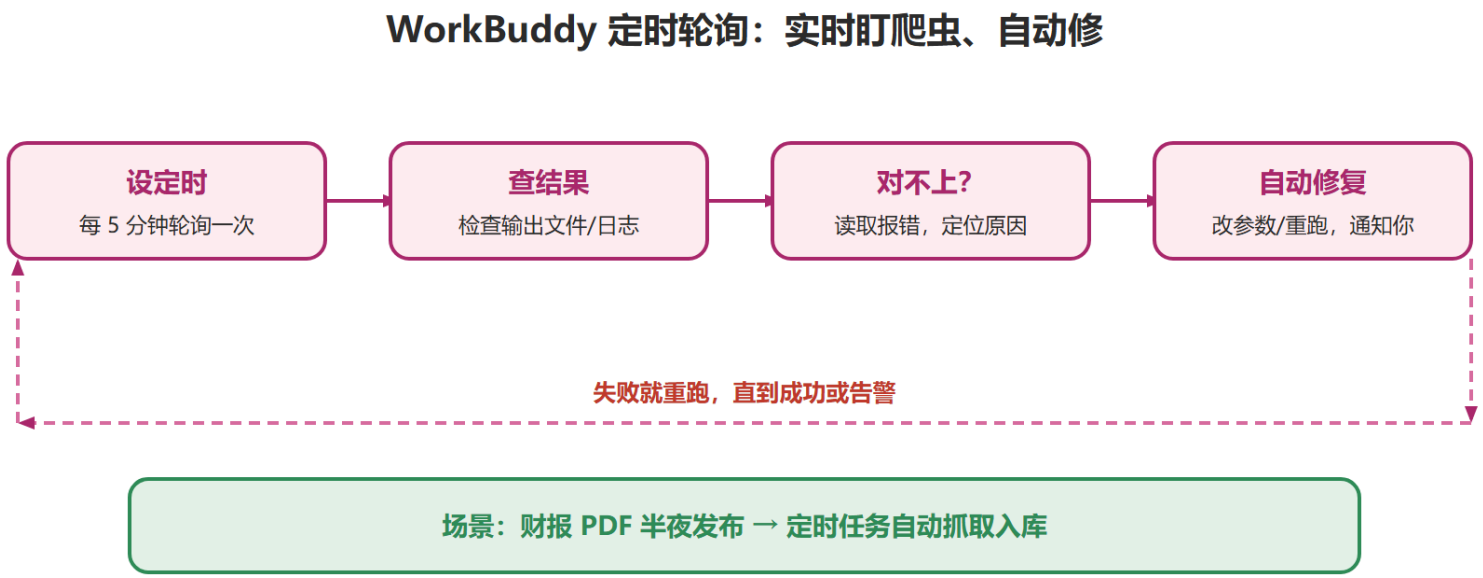
WorkBuddy 定时轮询：实时监控爬虫

爬虫跑起来后，不用人守着 —— 设个定时轮询

每 N 分钟检查一次：输出文件在不在、有没有报错

发现异常：读日志 → 定位原因 → 自动修复重跑

财报 PDF 半夜发布？定时任务自动抓取入库



★ 合规与公开边界

- **先看 robots.txt，人家不许抓的别抓**
- 控制请求频率，别把人家网站搞崩（记住 429）
- 哪些能随项目公开发布，哪些只能留脚本与字段说明
- 本项目最终要上 GitHub，这条边界今天就要想清楚

产出：一份带出处的数据表

- **每行一条记录，八个字段齐全**
- 额外三列：数据来源、获取时间、使用条款
- 采集失败的也要记下来，写进失败日志
- 这份表就是明天下午建模的直接输入

04

小结

三个板块串起来

小结：下午三个大板块

- **上接上午：数据为什么会脏，以及怎么洗干净**
- 可视化：用图发现数字背后的分布、异常与关系
- 数据获取：搞清数据从哪来、怎么来、什么能公开
- 明天的先导公开课会把这套数据接进模型